



WRITING BOOKS IN THE TERMINAL

The Inkhaven Manual

The Complete Guide to Writing a Book in the Terminal

Vladimir Ulogov

2026 · examples assume Inkhaven 3.0.0 or newer

Contents

What Inkhaven is	1
How this manual is organised	2
This manual, and the companions	2
A word on the pictures	3
Part I – Getting Started	4
1 – Installing Inkhaven	5
2 – Your First Project	6
3 – The Four Panes	7
Part II – Writing	8
4 – The Editor	9
5 – The Tree and Its Files	10
6 – Snapshots and History	11
7 – Search	12
8 – The Style Overlays	13
Part III – The AI Assistant	14
9 – The AI Assistant	15
10 – Chat With Your Book	16
11 – Reusable Prompts	17
12 – Watching the Cost	18
Part IV – The World & Its Facts	19
13 – Places and Characters	20
14 – The World	21
15 – The Knowledge Graph	22
16 – The Timeline	23
Part V – The Intelligences	24
17 – Continuity and Knowledge	25
18 – The Read-Through and the Voices	26
19 – Revision and History	27
20 – The Inner Family	28
Part VI – Language, Verse & Scholarship	29
21 – Constructed Languages	30
22 – Poetry	31
23 – Research and Scholarship	32
Part VII – Producing the Book	33
24 – Assembly and PDF	34

25 — EPUB, Web and More	35
26 — Technical Documentation	36
Part VIII — Scripting with Bund	37
27 — Scripting with Bund	38
Part IX — Keeping It Healthy	39
28 — Keeping It Healthy	40
29 — Configuration	41
Appendix A — The Keybinding Map	42
Appendix B — The Command Line	43
Appendix C — The Configuration Reference	44
Appendix D — The Feature Index	45
About the Author	46
Why Inkhaven exists	47
A work of love	47
A note on cooperation	48
Where to find more	48

About This Manual

Most writing tools ask you to leave the page. You open a browser to research, a second app to track your characters, a third to check continuity, a spreadsheet to count words, a design program to typeset. Inkhaven's premise is the opposite: **the whole of writing a book lives in one terminal window** — the prose, the world it is set in, the facts it must not contradict, the assistant you steer, and the finished PDF — and you never leave the keyboard to reach any of it.

This manual is the complete tour of that window. It assumes **no prior knowledge** — not of Inkhaven, not of the terminal beyond opening one — and it runs in reading order from installing the binary to publishing a book. Read it front to back once for the shape of the thing; return to any chapter when you reach the workflow it covers.

What Inkhaven is

Inkhaven is a single command-line program for writing books and long-form documentation. It pairs a full-screen editor for Typst — a modern typesetting language — with a local semantic index, an AI writing assistant you fully control, versioned snapshots, a backup pipeline, and an unusually deep set of **reading** intelligences that watch a manuscript for the things a long book gets wrong. Your prose lives as plain `.typ` files on disk; a local database tracks the hierarchy and the search index, but the words are always text you can read, diff, and version-control yourself.

TERM The manuscript is plain files

A paragraph is a `.typ` file on disk, inside a tree of folders for books, chapters, and subchapters. Inkhaven manages that tree and indexes it, but it never locks your words inside a proprietary format. You can open any paragraph in any editor, at any time, and Inkhaven will notice.

How this manual is organised

The book is in nine parts and a set of reference appendices:

● ● ● *The shape of the manual*

I	Getting Started	install, first project, the panes
II	Writing	the editor, the tree, snapshots, search
III	The AI Assistant	scopes, chat-with-your-book, prompts, cost
IV	The World & Facts	places, characters, the world, graph, timeline
V	The Intelligences	continuity, knowledge, read-through, voices,
		revision, the inner readers
VI	Language & Verse	conlang, poetry, research, scholarship
VII	Producing the Book ...	PDF, EPUB, web, technical docs
VIII	Scripting	the embedded Bund language
IX	Keeping It Healthy ...	backup, doctor, configuration
A–D	Reference	keys, commands, config, the feature index

Parts I through III are the ground floor — everything you need to write and get help. Parts IV and V are Inkhaven’s distinctive depth: the machinery for a book with a world, a cast, and secrets that must stay kept. Parts VI through IX are the specialist tracks and the plumbing.

This manual, and the companions

Inkhaven ships a small library of **companion books**, each a deep dive on one domain — **Know Your Book** on the reading intelligences, **Building the World** on worldbuilding, **Poetry with Inkhaven**, **Grounding Your Book in Fact** on research, and others. This manual is deliberately **breadth-first**: it tells you what

each feature is, how to reach it, and how to run it, then points you to the companion when you want the full treatment.

FICTION

If you write **fiction**, the heart of the book is Parts IV and V — a world and cast Inkhaven can hold in mind, and the readers that watch continuity, knowledge, voice, and pacing across a whole manuscript.

NON-FICTION

If you write **non-fiction**, your centre of gravity is the facts and the graph (Part IV), research and sources (Part VI), and the technical-doc and index tools (Part VII) — the machinery of claims that must be right.

A word on the pictures

Inkhaven is a terminal application, so this manual illustrates it with **terminal frames** rather than photographs — a faithful rendering of what you will see on screen, set in a monospace type. A frame is truer than a screenshot for a text interface, and it keeps the book self-contained. When you see one, read it as “this is what the screen shows here.”

WHAT YOU LEARNED

- Inkhaven is one terminal program for the **whole** of writing a book — prose, world, facts, an AI you steer, and the finished document.
- Your manuscript is always **plain .typ files** on disk; nothing is locked in a proprietary format.
- This manual is the **breadth-first tour**, install to publish; the companion books hold the deep dives.

PART I

Getting Started

CHAPTER 1

1

Installing Inkhaven

What Inkhaven needs, how to install it with Cargo, and what happens the first time you run it — including the one-time download of the local embedding model.

CHAPTER 2

2

Your First Project

`inkhaven init` and the anatomy of a project on disk: the `.typ` tree of Book to Chapter to Paragraph, the DuckDB metadata store, and the system books seeded for you.

CHAPTER 3

3

The Four Panes

The terminal UI at a glance — the Tree, the Editor, the AI pane, and the right-hand Output / Thoughts region — how focus moves between them, and the chord system that drives everything.

PART II

Writing

CHAPTER 4

4

The Editor

Moving, selecting, finding and replacing, undo and redo, and the split-edit view — the day-to-day mechanics of putting words on the page.

CHAPTER 5

5

The Tree and Its Files

Building and reshaping the manuscript tree, and the leaf types that live in it: prose, HJSON data, Jinja templates, Bund scripts, images, and the structural paragraph subtypes.

CHAPTER 6

6

Snapshots and History

Every paragraph's versioned history: taking and restoring F6 snapshots, the project-wide snapshot browser, and recovering deleted prose.

CHAPTER 7

7

Search

Finding any passage two ways — full-text and semantic (meaning-based, multilingual) — and how the local embedding index finds a paragraph that never used your exact words.

CHAPTER 8

8

The Style Overlays

The in-editor coaching overlays — filter-word crutches, echoed words, telling-not-showing, the POV chip, the sentence-rhythm gauge — each a toggle you question rather than obey.

PART III

The AI Assistant

CHAPTER 9

9

The AI Assistant

The AI pane end to end: choosing the scope, the local-versus-full knowledge mode, where the answer lands, and how six providers (and any the router reaches) plug in.

CHAPTER 10

10

Chat With Your Book

Retrieval-augmented conversation grounded in your own manuscript — the Book scope and the F9 structured scopes (Facts, Graph, and the reader conversations).

CHAPTER 11

11

Reusable Prompts

Writing prompt templates once and reusing them: the `prompts.hjson` library, the project-local Prompts book, the substitutions, and the picker.

CHAPTER 12

12

Watching the Cost

The cost dashboard and the daily caps — how Inkhaven keeps model spend visible and informs rather than surprises, and what each cost-capped feature actually costs.

PART IV

The World & Its Facts

CHAPTER 13

13

Places and Characters

The Places and Characters system books: recording your world's people and locations, the in-editor highlight overlays, and asking the AI about any of them.

CHAPTER 14

14

The World

Declaring a world's physics in `world.hjson` and letting the deterministic compiler derive its astronomy, geology, climate and demographics — plus maps and the live fact-checker.

CHAPTER 15

15

The Knowledge Graph

The typed-edge semantic net over your nodes, how it is populated, and chatting with it — walking the graph to answer a plain question about your own book.

CHAPTER 16

16

The Timeline

Recording events on a calendar of your own devising, and the timeline critique that catches orphans and impossible overlaps.

PART V

The Intelligences

CHAPTER 17

17

Continuity and Knowledge

The two continuity intelligences that watch a manuscript: SENTINEL (the book watches itself) and KEN (who knows what, when). How to run each and read its ledger.

CHAPTER 18

18

The Read-Through and the Voices

LECTOR reads the book forward as a first reader; CHORUS profiles the cast's voices; the dialogue and narrator-voice readers round out how it sounds.

CHAPTER 19

19

Revision and History

Turning findings into confirmed changes with REDLINE's Editorial Pass, and measuring whether a revision helped with CHRONICLE's draft history.

CHAPTER 20

20

The Inner Family

The readers who question rather than answer — Inner Socrates, Editor, Theologian, Poet, and the reasoning-rigor reader — and how to engage each.

PART VI

Language, Verse & Scholarship

CHAPTER 21

21

Constructed Languages

Building an invented language inside the editor — phonology, lexicon, morphology — and the WordNet thesaurus for real-world prose.

CHAPTER 22

22

Poetry

Measuring verse against a declared form — metre, rhyme, syllables — with a reader that observes and never generates.

CHAPTER 23

23

Research and Scholarship

Grounding a book in fact: the Research Assistant, the Sources bibliography, terminology governance, and the scholarly apparatus (loci, verborum, argument outline).

PART VII

Producing the Book

CHAPTER 24

24

Assembly and PDF

Turning the tree into a Typst-compilable directory and a finished PDF — the two-chord path from buffer to book.

CHAPTER 25

25

EPUB, Web and More

The other destinations: EPUB, an HTML website, the arXiv scientific bundle, DOCX, and the audiobook.

CHAPTER 26

26

Technical Documentation

The features for technical and reference books: the `docs` verification checks and the back-of-book index.

PART VIII

Scripting with Bund

CHAPTER 27

27

Scripting with Bund

The embedded scripting language: hook lambdas, the `ink.*` standard library, Bund Script nodes, and the sandbox policy that keeps them safe.

PART IX

Keeping It Healthy

CHAPTER 28

28

Keeping It Healthy

Backups and restore, the `reindex` command, the project doctor, and the advisory project lock — keeping the store sound over years of use.

CHAPTER 29

29

Configuration

A tour of `inkhaven.hjson` — how the layered config works, the blocks you are most likely to touch, and how to edit it safely from inside the editor.

APPENDIX A



The Keybinding Map

Every chord the TUI honours, by pane and overlay — the printable reference.
Mirrors the canonical `KEYBINDING.md`.

APPENDIX B

B

The Command Line

Every `inkhaven` subcommand at a glance, grouped by what it does.

APPENDIX C

C

The Configuration Reference

Every `inkhaven.hjson` block and field, with its default. Mirrors the canonical `CONFIGURATION.md`.

APPENDIX D

D

The Feature Index

The canonical map — every feature to its command, chord, doc, and scripting namespace.

AFTERWORD

About the author



Vladimir Ulogov.

Vladimir Ulogov has spent decades building infrastructure for distributed systems — the kind of software that watches other software. Early in his career he worked on monitoring and telemetry platforms; later years took him into federated observability, telemetry buses, and the architecture of systems that have to make sense of millions of data points without losing the thread.

Observability, in the end, is the art of **holding a system too large for one head** — of watching it without losing the thread, and never reporting a thing it cannot back up. It is not an accident that a tool he built for writers carries the same instinct to the desk: a book, past a certain size, is exactly such a system, and it too

deserves an instrument that holds it for you.

What makes him slightly unusual in his corner of the industry is a tendency to write his own tools — not in the sense of small utilities, but in the sense of programming languages. The Bund language (its compiler, its VM, its document store, its parser) lives in a long series of Rust crates on crates.io. `rust_dynamic`, `rust_multistack`, `rust_multistackvm`, `bundcore`, `zbus_universal_data_gateway` — each is a building block that exists because the off-the-shelf options didn't fit the shape of the work.

Why Inkhaven exists

Inkhaven is Vladimir's personal reflection on how a single tool can hold the **whole** of writing a book. The frustration it answers is one every long-form writer knows: that the work is scattered across a dozen apps that don't talk to each other — a browser for research, one program for the prose, another for the characters, a spreadsheet for the word count, a design tool for the final typesetting — and every seam between them is a place the book leaks.

Inkhaven's wager is that all of it belongs in one place, next to the keyboard: the prose and the world it is set in, the facts it must not contradict, the assistant you steer, and the finished PDF. Not because integration is elegant, but because a book past a certain size outgrows the mind writing it, and a tool that holds your words should also be able to **hold the book** — its facts, its continuity, its shape — so the writer is free to do the one thing no tool can, which is write.

A work of love

Inkhaven is open source. The licence is permissive — you can read it, fork it, study it, modify it, and pass it on. Strictly speaking, the licence also lets you sell it. The author would, gently but firmly, disagree with you doing that. Inkhaven was not designed as a *for sale* project. It is a work of love made for the authors who can least afford to pay for software, and turning it into a commercial product would betray the reason it exists. So please — don't.

It carries no analytics, no telemetry, no upsell. The binary will never phone home; the project will never have an “Enterprise” tier you have to escape from.

This was a deliberate choice. There are excellent commercial tools for writing. They cost money — which is fine for many writers, and a barrier for many more. Inkhaven exists for the second group: for the graduate student writing a dissertation on a battered laptop, for the novelist who shouldn't have to pick between rent and software, for the engineer drafting in the same terminal where they already write code, for anyone who would benefit from a tool that respects their work without asking for a credit-card number.

A note on cooperation

Vladimir believes firmly in the human capacity for mutual help — that we make better work, and live better lives, when we share what we know and what we build. Open source is one of the most concrete expressions of cooperation our era has produced: code read, improved, and passed forward without payment, without permission, by people who will never meet.

If Inkhaven helps you finish your book — that is enough. If it gives you a chord pattern you adapt into your own tool — that's a gift back to the larger project of making software writers can love. As a society, we achieve the greatest things when we help each other rather than compete with each other.

Where to find more

GitHub	@vulogov — the source for Inkhaven, Bund, and the dozen-plus Rust crates that carry the infrastructure. Issues and PRs welcome.
LinkedIn	/in/vladimirulogov — posts on observability, the occasional long-form essay.
YouTube	@vulogov — talks and walkthroughs from the conference trail.
X / Twitter	@vladimir_ulogov

If a fact you grounded ever surprises you, or a command trips you up and you can't find the answer in this book — open an issue on GitHub. The author reads them.

THE INKHAVEN MANUAL

END OF THE BOOK · INKHAVEN 3.0.0